Symbols, Iterators & Proxy

These three features are how JavaScript exposes its own internals for extension. `Symbol` gives you collision-proof keys and a set of "well-known" hooks the language itself reads; `Symbol.iterator` is the specific hook that makes an object usable with `for ... of` and spread; and `Proxy` (with its partner `Reflect`) lets you intercept fundamental operations like property reads and writes. Interviewers reach for them because they separate people who use the language from people who understand its object model — and because Proxy powers real tools you have used, like Vue 3's reactivity.

Symbol: a guaranteed-unique primitive

A `Symbol` is a primitive value (like a number or string) whose only guarantee is uniqueness: every `Symbol()` call returns a brand-new value that is not equal to any other, even one created with the same description. The description is just a debugging label, not an identity.

JS Copy

```
const s1 = Symbol('id');
const s2 = Symbol('id');
console.log(s1 === s2);        // ⇒ false
console.log(s1.description);   // ⇒ "id"
```

The primary use is as a **non-colliding object key**. If two libraries both want to stash metadata on your object under a string key like `"id"`, they clobber each other. A symbol key can't collide with anything else, because no one else holds that exact symbol.

private (they're reachable via `Object.getOwnPropertySymbols`), but they stay out of the way of generic iteration and serialization.

JS Copy

```
const user = { name: 'Ada', [Symbol('secret')]: 42 };
for (const k in user) console.log(k);                // ⇒ "name"  (only)
console.log(JSON.stringify(user));                   // ⇒ {"name":"Ada"}
console.log(Object.keys(user));                      // ⇒ ["name"]
console.log(Object.getOwnPropertySymbols(user).length); // ⇒ 1
```

Symbol.for — the global registry

`Symbol()` is always fresh. When you *do* want two pieces of code to obtain the *same* symbol by a shared name — across modules or even across realms like iframes — use `Symbol.for(key)`. It looks the key up in a process-wide registry and returns the same symbol every time. `Symbol.keyFor` recovers the registry key (and returns `undefined` for a non-registered local symbol).

JS Copy

```
const g1 = Symbol.for('app.id');
const g2 = Symbol.for('app.id');
console.log(g1 === g2);               // ⇒ true
console.log(Symbol.keyFor(g1));       // ⇒ "app.id"
console.log(Symbol.keyFor(Symbol('id'))); // ⇒ undefined
```

Well-known symbols

The language keeps a set of built-in symbols on `Symbol` itself that act as **extension hooks**: put a method under one of these keys and you customize how the engine treats your object. The ones worth knowing:

- `Symbol.iterator` — makes an object iterable (`for … of`, spread, destructuring). Covered below.
- `Symbol.asyncIterator` — the async equivalent, consumed by `for await … of`.

- `Symbol.hasInstance` — customizes what `x instanceof Ctor` returns.

```js
const money = {
  amount: 100,
  [Symbol.toPrimitive](hint) {
    if (hint === 'number') return this.amount;    // +money, money * 2
    if (hint === 'string') return `$${this.amount}`; // `${money}`
    return `Money(${this.amount})`;               // money + '', money + 1
  }
};
console.log(+money);          // ⇒ 100        (number hint)
console.log(`${money}`);      // ⇒ "$100"     (string hint)
console.log(money + 1);       // ⇒ "Money(100)1"  (default hint)
```

`Symbol.hasInstance` lets a "class" answer `instanceof` however it likes — here without any prototype chain at all:

```js
class Even {
  static [Symbol.hasInstance](n) { return n % 2 === 0; }
}
console.log(4 instanceof Even); // ⇒ true
console.log(3 instanceof Even); // ⇒ false
```

Iterator protocol via Symbol.iterator

An object is **iterable** if it has a `[Symbol.iterator]()` method returning an **iterator**: an object with a `next()` that yields `{ value, done }` on each call. `for ... of`, spread `[...x]`, `Array.from`, and array destructuring all speak this protocol. (Generators, covered in their own topic, are just a terser way to *produce* one of these iterators — here we build it by hand to see the shape.)

```js
```

```
  to: 4,
  [Symbol.iterator]() {
    let current = this.from;
    const last = this.to;
    return {
      next() {
        return current ≤ last
          ? { value: current++, done: false }
          : { value: undefined, done: true };
      }
    };
  }
};
console.log([...range]);        // ⇒ [1, 2, 3, 4]
const [a, b] = range;
console.log(a, b);              // ⇒ 1 2
```

The two roles are distinct and interviewers test the distinction: the **iterable** hands out iterators (via `[Symbol.iterator]`); the **iterator** produces values (via `next`). Returning a fresh iterator each time makes the object re-iterable — reuse one stateful iterator and the second `for ... of` finds it already exhausted.

Proxy: intercepting fundamental operations

A `Proxy` wraps a target object and lets you install **traps** — handler functions that intercept fundamental operations. Read a property, write one, check `in`, `delete`, or call the object as a function, and the engine routes that operation through your handler instead of straight to the target. The common traps: `get`, `set`, `has` (the `in` operator), `deleteProperty`, and `apply` (calling a function target).

This one example combines defaulting on read, validation on write, and hiding underscore-prefixed keys from `in`:

JS Copy

```
const user = new Proxy({}, {
  get(target, prop, receiver) {
```

```
  set(target, prop, value, receiver) {
    if (prop === 'age' && (typeof value !== 'number' || value < 0)) {
      throw new TypeError('age must be a non-negative number');
    }
    return Reflect.set(target, prop, value, receiver);
  },
  has(target, prop) {                                    // the `in` op
    if (typeof prop === 'string' && prop.startsWith('_')) return false;
    return Reflect.has(target, prop);
  }
});

user.name = 'Ada';
console.log(user.name);        // => "Ada"
console.log(user.email);       // => "<no email>"   (get trap default)
user._secret = 1;
console.log('_secret' in user); // => false         (has trap hides it)
try { user.age = -5; } catch (e) { console.log(e.message); } // => "age mu
user.age = 30;
console.log(user.age);         // => 30
```

Two more classic tricks. Negative array indices (Python-style `arr[-1]`) via the `get` trap:

JS Copy

```
function negArray(arr) {
  return new Proxy(arr, {
    get(target, prop, receiver) {
      const i = Number(prop);
      if (typeof prop === 'string' && Number.isInteger(i) && i < 0) {
        return Reflect.get(target, String(target.length + i), receiver);
      }
      return Reflect.get(target, prop, receiver);
    }
  });
}
```

And the `apply` trap, which fires when a function target is *called* — a transparent logging or timing wrapper:

JS Copy

```
function sum(x, y) { return x + y; }
const logged = new Proxy(sum, {
  apply(target, thisArg, args) {
    console.log('called with:', args);
    return Reflect.apply(target, thisArg, args);
  }
});
console.log(logged(2, 3)); // logs "called with: [ 2, 3 ]", then ⟹ 5
```

Reflect: the companion to Proxy

`Reflect` is a namespace object whose methods mirror the proxy traps one-for-one: `Reflect.get`, `Reflect.set`, `Reflect.has`, `Reflect.deleteProperty`, `Reflect.apply`, and so on. It gives you a function-call way to perform the *default* behaviour of the operation you just intercepted, so inside a trap you do your custom bit and then delegate the rest to `Reflect.*`.

Why not just write `target[prop]` inside a `get` trap? Two reasons. First, `Reflect` methods return a **meaningful value** and never throw where an operator would: `Reflect.set` returns a boolean instead of silently swallowing failures, which is exactly the shape the trap must return. Second, and more important, `Reflect.get(target, prop, receiver)` forwards the `receiver` — the object the operation was originally invoked on. That keeps `this` correct when a getter on the prototype runs through the proxy. Reaching for `target[prop]` instead loses the receiver and can bind getters to the wrong object. Trap plus `Reflect` is the idiomatic pair: intercept, then forward faithfully.

Real-world uses and cost

- **Reactive frameworks.** Vue 3's reactivity is a `Proxy` over your state: the `get` trap records which effect is reading a property (dependency tracking), and the `set` trap re-runs the effects that depend on it. MobX offers a similar proxy-based mode. This is the headline "what is Proxy for" answer.
- **Validation / invariants.** Enforce schema, types, or read-only-ness at assignment time via the `set` trap, as above.
- **Logging, tracing, mocking.** The `apply` and `get` traps make transparent instrumentation and test spies without touching the wrapped code.
- **Defaulting and virtual properties.** The `get` trap can synthesize values that don't physically exist — default values, computed API surfaces, or negative indexing.

The cost is real, so mention it: every trapped operation is a function call, so a proxy is meaningfully slower than a plain object on hot paths and is **not optimized by the JIT** the way ordinary property access is. Traps also don't fire on operations that bypass them (an inner reference to the raw target skips the proxy entirely), and there are invariants the engine enforces — a `get` trap must not report a different value for a non-writable, non-configurable data property, or it throws. Use proxies for framework-level plumbing, not inside tight numeric loops.

Common interview questions

Q: Why do Symbols exist / what problem do they solve? They provide guaranteed-unique property keys, so independent code can attach metadata to an object without colliding on string names, and those keys stay out of `for ... in` / `Object.keys` / `JSON`. They also serve as the language's extension hooks (the well-known symbols).

Q: What's the difference between `Symbol()` and `Symbol.for()`? `Symbol()` always returns a brand-new unique symbol. `Symbol.for(key)` returns a *shared* symbol from a global registry keyed by string, so repeated calls with the same key — even in different modules — return the identical symbol.

Q: What makes an object iterable? It has a `[Symbol.iterator]()` method returning an iterator (an object with `next()` yielding `{ value, done }`). That's what `for ... of`, spread, and destructuring consume. Arrays, strings, `Map`, and `Set` are iterable; plain objects are not.

state (Vue 3), validation, logging, and virtual/defaulting properties.

Q: Why does a Proxy trap pair with `Reflect`? `Reflect.*` performs the default operation and, crucially, forwards the `receiver`, so getters/setters keep the correct `this`; it also returns the boolean/value the trap is expected to return, unlike raw operators.

Q: Can Symbols be used as WeakMap keys? No — `WeakMap` keys must be objects (registered symbols are a recent, narrow exception in newer engines, but the safe interview answer is "no, symbols are primitives").

Gotchas / traps

Symbols do not auto-coerce to string. Concatenating or interpolating a symbol throws a `TypeError` — you must convert explicitly with `String(sym)` or `sym.toString()`.

JS Copy

```
const sym = Symbol('x');
try { '' + sym; } catch (e) { console.log(e.name); } // ⇒ "TypeError"
console.log(String(sym));                            // ⇒ "Symbol(x)"
// `${sym}` throws for the same reason.
```

Symbol keys survive `Object.assign` and spread but vanish from JSON. `{ ...obj }` and `Object.assign` copy symbol-keyed properties, yet `JSON.stringify` drops them silently. Round-tripping an object through JSON loses all symbol data — a real bug when symbols hold important metadata.

A Proxy is not `===` its target. Wrapping changes identity, so any code that compares against the original object, or uses the target as a `Map`/`Set` key, will not match the proxy.

JS Copy

```
const obj = {};
const p = new Proxy(obj, {});
```

Proxies and `#private` fields / internal slots don't mix. A private field is keyed to the *actual* instance, not the proxy. Call a method through the proxy and, unless the trap unwraps to the real target, accessing `this.#field` throws because `this` is the proxy, which has no such field. The same "wrong internal slot" problem hits built-ins like `Map`, `Date`, and `Set` when proxied.

JS Copy

```
class Box { #v = 7; get() { return this.#v; } }
const pbox = new Proxy(new Box(), {});
try { pbox.get(); } catch (e) { console.log(e.name); } // ⇒ "TypeError"
```

`for ... in` reaches inherited enumerable string keys but never symbols. Interviewers pair this with the symbol-hiding point: symbol keys are skipped by `for ... in` regardless of enumerability; to read them you need `Object.getOwnPropertySymbols` or `Reflect.ownKeys` (which returns strings *and* symbols).

QUICK CHECK — no marks, just for you.

1. What does this log?

JAVASCRIPT Copy

```
const user = new Proxy({}, {
  get(target, prop) {
    if (!(prop in target)) return `<no ${String(prop)}>`;
    return target[prop];
  },
  set(target, prop, value) {
    if (prop === 'age' && (typeof value !== 'number' || value < 0)) {
      throw new TypeError('bad age');
    }
    target[prop] = value;
    return true;
  }
});
user.name = 'Ada';
```

- Ada
  undefined
- Ada
  <no undefined>
- Ada
  <no email>
- undefined
  <no email>

Check answer

2. What does this log?

JAVASCRIPT Copy

```
const money = {
  amount: 100,
  [Symbol.toPrimitive](hint) {
    if (hint === 'number') return this.amount;
    if (hint === 'string') return `$${this.amount}`;
    return `Money(${this.amount})`;
  }
};
console.log(+money);
console.log(`${money}`);
console.log(money + 1);
```

- 100
  $100
  Money(100)1
- 100
  $100
  101

```
$100
$100
Money(100)1
```

Check answer

3. Why do Symbols exist / what core problem do they solve as object keys?

- They make property values immutable so the object cannot be mutated after creation.
- They make a property completely private and unreachable by any code outside the object.
- They are a faster, hashed alternative to string keys for large objects.
- They provide guaranteed-unique keys so independent code can attach metadata without colliding on string names, and they stay out of for...in / Object.keys / JSON.

Check answer

Previous
Modules: ESM vs CommonJS

Next
Memory Management & Garbage Collection

LEARN MORE

ARTICLES & DOCS

Proxy (MDN)
MDN

VIDEOS



Proxy and Reflect objects

YouTube



Proxy object in ES6

YouTube



Reflect & Proxy for beginners

YouTube

Hand-picked references for this lesson.